

Formative Assignment: Advanced Linear Algebra (PCA)

This notebook implements Principal Component Analysis from scratch using only NumPy.

Dataset: Africa Development Indicators — 12 Sub-Saharan African countries, 2010–2022

Features: GDP per capita, Literacy Rate, Life Expectancy, Infant Mortality, Access to Clean Water, School Enrollment, Health Expenditure, Urban Population

Group: Group 13

Group Members: Hirwa Hebert Crispin · Mildred Ewenrim Ebomah

GitHub: <https://github.com/Hirwavsopps/PCA-Africa-Formative2>

Step 0: Load the Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# — CREATE DATASET DIRECTLY – no CSV upload needed —————
np.random.seed(42)
n = 120
countries =
['Rwanda', 'Kenya', 'Uganda', 'Tanzania', 'Ethiopia', 'Burundi',
'DRC', 'Zambia', 'Malawi', 'Mozambique', 'Zimbabwe', 'Nigeria']
regions = ['East Africa', 'East Africa', 'East Africa', 'East Africa',
           'East Africa', 'East Africa', 'Central Africa', 'Southern
Africa',
           'Southern Africa', 'Southern Africa', 'Southern
Africa', 'West Africa']
idx = np.random.randint(0, len(countries), n)
data = {
    'Country': [countries[i] for i in idx],
    'Region': [regions[i] for i in idx],
    'Year': np.random.randint(2010, 2023, n).tolist(),
    'GDP_per_capita_USD':
np.random.normal(900, 400, n).clip(200, 3000).round(1).tolist(),
    'Literacy_Rate_pct':
np.random.normal(68, 15, n).clip(30, 98).round(1).tolist(),
    'Life_Expectancy_yrs':
np.random.normal(62, 7, n).clip(45, 78).round(1).tolist(),
    'Infant_Mortality_per1k':
np.random.normal(45, 18, n).clip(10, 95).round(1).tolist(),
    'Access_Clean_Water_pct':
np.random.normal(58, 20, n).clip(15, 95).round(1).tolist(),
    'School_Enrollment_pct':
```

```

np.random.normal(72,18,n).clip(30,99).round(1).tolist(),
    'Health_Expenditure_pct':
np.random.normal(5.2,2.1,n).clip(1.0,12.0).round(2).tolist(),
    'Urban_Population_pct':
np.random.normal(30,15,n).clip(8,75).round(1).tolist(),
}
df = pd.DataFrame(data)
for col in
['GDP_per_capita_USD', 'Literacy_Rate_pct', 'Infant_Mortality_per1k',
    'Access_Clean_Water_pct', 'Health_Expenditure_pct']:
    mask = np.random.random(n) < 0.09
    df.loc[mask, col] = np.nan
df.to_csv('africa_development_indicators.csv', index=False)
# -----

print('Dataset Shape:', df.shape)
print('\nFirst 5 rows:')
df.head()

```

Dataset Shape: (120, 11)

First 5 rows:

```

{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 120,\n  \"fields\": [\n    {\n      \"column\": \"Country\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 12,\n        \"samples\": [\n          \"Rwanda\",\n          \"Nigeria\",\n          \"DRC\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Region\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 4,\n        \"samples\": [\n          \"East Africa\",\n          \"West Africa\",\n          \"Central Africa\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Year\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 3,\n        \"min\": 2010,\n        \"max\": 2022,\n        \"num_unique_values\": 13,\n        \"samples\": [\n          2022,\n          2020,\n          2018\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"GDP_per_capita_USD\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 383.7725410591904,\n        \"min\": 200.0,\n        \"max\": 1993.8,\n        \"num_unique_values\": 108,\n        \"samples\": [\n          920.5,\n          1269.3,\n          1209.3\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Literacy_Rate_pct\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 15.183386184028398,\n        \"min\": 36.6,\n        \"max\": 98.0,\n        \"num_unique_values\": 95,\n        \"samples\": [\n          50.1,\n          96.1,\n          36.6\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    ]\n  ]\n}

```

```

}\n    },\n    {\n        \"column\": \"Life_Expectancy_yrs\",\n        \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 7.336783502111996,\n            \"min\": 45.0,\n            \"max\": 78.0,\n            \"num_unique_values\": 98,\n            \"samples\": [\n                58.6,\n                65.7,\n                62.2\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n        },\n        {\n            \"column\": \"Infant_Mortality_per1k\",\n            \"properties\": {\n                \"dtype\": \"number\",\n                \"std\": 17.108566642000913,\n                \"min\": 10.0,\n                \"max\": 88.8,\n                \"num_unique_values\": 102,\n                \"samples\": [\n                    35.2,\n                    39.3,\n                    73.4\n                ],\n                \"semantic_type\": \"\",\n                \"description\": \"\"\n            },\n            {\n                \"column\": \"Access_Clean_Water_pct\",\n                \"properties\": {\n                    \"dtype\": \"number\",\n                    \"std\": 20.74674391748185,\n                    \"min\": 15.0,\n                    \"max\": 95.0,\n                    \"num_unique_values\": 94,\n                    \"samples\": [\n                        61.2,\n                        46.3,\n                        68.6\n                    ],\n                    \"semantic_type\": \"\",\n                    \"description\": \"\"\n                },\n                {\n                    \"column\": \"School_Enrollment_pct\",\n                    \"properties\": {\n                        \"dtype\": \"number\",\n                        \"std\": 17.657828327982582,\n                        \"min\": 30.0,\n                        \"max\": 99.0,\n                        \"num_unique_values\": 102,\n                        \"samples\": [\n                            66.0,\n                            77.1,\n                            79.4\n                        ],\n                        \"semantic_type\": \"\",\n                        \"description\": \"\"\n                    },\n                    {\n                        \"column\": \"Health_Expenditure_pct\",\n                        \"properties\": {\n                            \"dtype\": \"number\",\n                            \"std\": 1.8204469674154646,\n                            \"min\": 1.0,\n                            \"max\": 10.15,\n                            \"num_unique_values\": 100,\n                            \"samples\": [\n                                4.72,\n                                2.17,\n                                6.56\n                            ],\n                            \"semantic_type\": \"\",\n                            \"description\": \"\"\n                        },\n                        {\n                            \"column\": \"Urban_Population_pct\",\n                            \"properties\": {\n                                \"dtype\": \"number\",\n                                \"std\": 14.14951128017456,\n                                \"min\": 8.0,\n                                \"max\": 74.5,\n                                \"num_unique_values\": 99,\n                                \"samples\": [\n                                    16.5,\n                                    31.7,\n                                    35.7\n                                ],\n                                \"semantic_type\": \"\",\n                                \"description\": \"\"\n                            }\n                        }\n                    }\n                },\n                {\n                    \"type\": \"dataframe\", \"variable_name\": \"df\"\n                }\n            }\n        }\n    }\n}

```

```
import requests
```

```
url =
\"https://raw.githubusercontent.com/datasets/gdp/master/data/gdp.csv\"
```

```
# Alternative: just recreate the dataset directly in Colab
```

```
import numpy as np
import pandas as pd
```

```
np.random.seed(42)
```

```
n = 120
```

```
countries =
```

```
['Rwanda', 'Kenya', 'Uganda', 'Tanzania', 'Ethiopia', 'Burundi',
```

```

'DRC','Zambia','Malawi','Mozambique','Zimbabwe','Nigeria']
regions = ['East Africa','East Africa','East Africa','East Africa',
           'East Africa','East Africa','Central Africa','Southern
Africa',
           'Southern Africa','Southern Africa','Southern Africa','West
Africa']

idx = np.random.randint(0, len(countries), n)
data = {
    'Country': [countries[i] for i in idx],
    'Region': [regions[i] for i in idx],
    'Year': np.random.randint(2010, 2023, n),
    'GDP_per_capita_USD':
np.random.normal(900,400,n).clip(200,3000).round(1),
    'Literacy_Rate_pct':
np.random.normal(68,15,n).clip(30,98).round(1),
    'Life_Expectancy_yrs':
np.random.normal(62,7,n).clip(45,78).round(1),
    'Infant_Mortality_per1k':
np.random.normal(45,18,n).clip(10,95).round(1),
    'Access_Clean_Water_pct':
np.random.normal(58,20,n).clip(15,95).round(1),
    'School_Enrollment_pct':
np.random.normal(72,18,n).clip(30,99).round(1),
    'Health_Expenditure_pct':
np.random.normal(5.2,2.1,n).clip(1.0,12.0).round(2),
    'Urban_Population_pct':
np.random.normal(30,15,n).clip(8,75).round(1),
}

# Inject missing values
df = pd.DataFrame(data)
for col in
['GDP_per_capita_USD','Literacy_Rate_pct','Infant_Mortality_per1k',
 'Access_Clean_Water_pct','Health_Expenditure_pct']:
    mask = np.random.random(n) < 0.09
    df.loc[mask, col] = np.nan

df.to_csv('africa_development_indicators.csv', index=False)
print("Dataset created. Shape:", df.shape)
df.head()

```

Dataset created. Shape: (120, 11)

```

{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 120,\n  \"fields\": [\n    {\n      \"column\": \"Country\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 12,\n        \"samples\": [\n          \"Rwanda\",\n          \"Nigeria\",\n          \"DRC\"\n        ],\n        \"semantic_type\": \"\",

```

```

"description\": \"\"\" } \n }, \n { \n \"column\":
\"Region\", \n \"properties\": { \n \"dtype\":
\"category\", \n \"num_unique_values\": 4, \n \"samples\":
[ \n \"East Africa\", \n \"West Africa\", \n
\"Central Africa\" \n ], \n \"semantic_type\": \"\", \n
\"description\": \"\"\" } \n }, \n { \n \"column\":
\"Year\", \n \"properties\": { \n \"dtype\": \"number\", \n
\"std\": 3, \n \"min\": 2010, \n \"max\": 2022, \n
\"num_unique_values\": 13, \n \"samples\": [ \n 2022, \n
2020, \n 2018 \n ], \n \"semantic_type\": \"\", \n
\"description\": \"\"\" } \n }, \n { \n \"column\":
\"GDP_per_capita_USD\", \n \"properties\": { \n \"dtype\":
\"number\", \n \"std\": 383.7725410591904, \n \"min\":
200.0, \n \"max\": 1993.8, \n \"num_unique_values\": 108, \n
\"samples\": [ \n 920.5, \n 1269.3, \n
1209.3 \n ], \n \"semantic_type\": \"\", \n
\"description\": \"\"\" } \n }, \n { \n \"column\":
\"Literacy_Rate_pct\", \n \"properties\": { \n \"dtype\":
\"number\", \n \"std\": 15.183386184028398, \n \"min\":
36.6, \n \"max\": 98.0, \n \"num_unique_values\": 95, \n
\"samples\": [ \n 50.1, \n 96.1, \n 36.6 \n
], \n \"semantic_type\": \"\", \n \"description\": \"\"\"
} \n }, \n { \n \"column\": \"Life_Expectancy_yrs\", \n
\"properties\": { \n \"dtype\": \"number\", \n \"std\":
7.336783502111996, \n \"min\": 45.0, \n \"max\": 78.0, \n
\"num_unique_values\": 98, \n \"samples\": [ \n 58.6, \n
65.7, \n 62.2 \n ], \n \"semantic_type\": \"\", \n
\"description\": \"\"\" } \n }, \n { \n \"column\":
\"Infant_Mortality_per1k\", \n \"properties\": { \n
\"dtype\": \"number\", \n \"std\": 17.108566642000913, \n
\"min\": 10.0, \n \"max\": 88.8, \n \"num_unique_values\":
102, \n \"samples\": [ \n 35.2, \n 39.3, \n
73.4 \n ], \n \"semantic_type\": \"\", \n
\"description\": \"\"\" } \n }, \n { \n \"column\":
\"Access_Clean_Water_pct\", \n \"properties\": { \n
\"dtype\": \"number\", \n \"std\": 20.74674391748185, \n
\"min\": 15.0, \n \"max\": 95.0, \n \"num_unique_values\":
94, \n \"samples\": [ \n 61.2, \n 46.3, \n
68.6 \n ], \n \"semantic_type\": \"\", \n
\"description\": \"\"\" } \n }, \n { \n \"column\":
\"School_Enrollment_pct\", \n \"properties\": { \n
\"dtype\": \"number\", \n \"std\": 17.657828327982582, \n
\"min\": 30.0, \n \"max\": 99.0, \n \"num_unique_values\":
102, \n \"samples\": [ \n 66.0, \n 77.1, \n
79.4 \n ], \n \"semantic_type\": \"\", \n
\"description\": \"\"\" } \n }, \n { \n \"column\":
\"Health_Expenditure_pct\", \n \"properties\": { \n
\"dtype\": \"number\", \n \"std\": 1.8204469674154646, \n
\"min\": 1.0, \n \"max\": 10.15, \n \"num_unique_values\":

```

```

100,\n          \"samples\": [\n          4.72,\n          2.17,\n6.56\n          ],\n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          }\n          },\n          {\n          \"column\": \n          \"Urban_Population_pct\", \n          \"properties\": {\n          \"dtype\": \n          \"number\", \n          \"std\": 14.14951128017456, \n          \"min\": \n          8.0, \n          \"max\": 74.5, \n          \"num_unique_values\": 99, \n          \"samples\": [\n          16.5, \n          31.7, \n          35.7\n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n          }\n          }\n          ],\n          \"type\": \"dataframe\", \"variable_name\": \"df\"}

```

Inspect data types and missing values

```

print("Column Data Types:")
print(df.dtypes)
print("\nMissing Values per Column:")
print(df.isnull().sum())

```

Column Data Types:

Country	object
Region	object
Year	int64
GDP_per_capita_USD	float64
Literacy_Rate_pct	float64
Life_Expectancy_yrs	float64
Infant_Mortality_per1k	float64
Access_Clean_Water_pct	float64
School_Enrollment_pct	float64
Health_Expenditure_pct	float64
Urban_Population_pct	float64
dtype:	object

Missing Values per Column:

Country	0
Region	0
Year	0
GDP_per_capita_USD	12
Literacy_Rate_pct	10
Life_Expectancy_yrs	0
Infant_Mortality_per1k	10
Access_Clean_Water_pct	12
School_Enrollment_pct	0
Health_Expenditure_pct	15
Urban_Population_pct	0
dtype:	int64

Step 1: Load and Standardize the Data

Before applying PCA, we must:

1. Handle non-numeric columns (Country, Region) using label encoding
2. Impute missing values with the column mean

3. Standardize all features: $z = (x - \mu) / \sigma$

```
# --- Handle non-numeric columns using label encoding ---
from numpy import unique

# Encode 'Country' column
countries = df['Country'].values
unique_countries, country_encoded = unique(countries,
return_inverse=True)
df['Country_encoded'] = country_encoded

# Encode 'Region' column
regions = df['Region'].values
unique_regions, region_encoded = unique(regions, return_inverse=True)
df['Region_encoded'] = region_encoded

print("Country encoding mapping:")
for i, c in enumerate(unique_countries):
    print(f" {c} -> {i}")

print("\nRegion encoding mapping:")
for i, r in enumerate(unique_regions):
    print(f" {r} -> {i}")

Country encoding mapping:
Burundi -> 0
DRC -> 1
Ethiopia -> 2
Kenya -> 3
Malawi -> 4
Mozambique -> 5
Nigeria -> 6
Rwanda -> 7
Tanzania -> 8
Uganda -> 9
Zambia -> 10
Zimbabwe -> 11

Region encoding mapping:
Central Africa -> 0
East Africa -> 1
Southern Africa -> 2
West Africa -> 3

# --- Select numeric feature columns for PCA ---
feature_cols = [
    'GDP_per_capita_USD',
    'Literacy_Rate_pct',
    'Life_Expectancy_yrs',
    'Infant_Mortality_per1k',
    'Access_Clean_Water_pct',
```

```

    'School_Enrollment_pct',
    'Health_Expenditure_pct',
    'Urban_Population_pct',
    'Country_encoded',
    'Region_encoded'
]

X = df[feature_cols].values # shape: (120, 10)
print("Feature matrix shape:", X.shape)

Feature matrix shape: (120, 10)

# --- Impute missing values with column mean (numpy only) ---
col_means = np.nanmean(X, axis=0)

# Find indices where values are NaN and replace with column mean
nan_mask = np.isnan(X)
X_imputed = X.copy()
for col_idx in range(X.shape[1]):
    X_imputed[nan_mask[:, col_idx], col_idx] = col_means[col_idx]

print("Missing values after imputation:", np.isnan(X_imputed).sum())
print("\nFirst 5 rows after imputation:")
print(X_imputed[:5])

Missing values after imputation: 0

First 5 rows after imputation:
[[1.0359e+03 8.8500e+01 6.5300e+01 4.1600e+01 8.5700e+01 9.9000e+01
 3.7200e+00 8.0000e+00 1.0000e+00 0.0000e+00]
 [9.7095e+02 8.0400e+01 6.2400e+01 4.4100e+01 7.8200e+01 8.4900e+01
 3.7200e+00 3.8500e+01 8.0000e+00 1.0000e+00]
 [1.2069e+03 6.8200e+01 7.7900e+01 2.5900e+01 2.5400e+01 6.3600e+01
 1.0150e+01 2.0500e+01 1.1000e+01 2.0000e+00]
 [4.4010e+02 7.4600e+01 6.4800e+01 3.3200e+01 6.3600e+01 8.1500e+01
 7.1700e+00 4.4900e+01 1.0000e+01 2.0000e+00]
 [5.8990e+02 8.3700e+01 5.9000e+01 3.1100e+01 3.6800e+01 9.9000e+01
 3.4400e+00 2.5800e+01 2.0000e+00 1.0000e+00]]

# Step 1: Load and Standardize the data (use of numpy only allowed)
# Formula:  $z = (x - \text{mean}) / \text{std}$ 

mean = np.mean(X_imputed, axis=0) # mean of each feature
std = np.std(X_imputed, axis=0) # std of each feature

standardized_data = (X_imputed - mean) / std # Do not use sklearn

# Verify standardization: mean  $\approx 0$ , std  $\approx 1$ 
print("Mean after standardization (should be ~0):")
print(np.round(np.mean(standardized_data, axis=0), 4))
print("\nStd after standardization (should be ~1):")

```

```
print(np.round(np.std(standardized_data, axis=0), 4))

standardized_data[:5]

Mean after standardization (should be ~0):
[-0.  0.  0. -0.  0. -0. -0.  0. -0. -0.]

Std after standardization (should be ~1):
[1.  1.  1.  1.  1.  1.  1.  1.  1.  1.]

array([[ 0.17922723,  1.26487065,  0.42247515, -0.15666695, -
 1.52878219,
        1.61950284, -0.94589373, -1.67430404, -1.42207395, -
 1.85592145],
       [ 0.          ,  0.7051211 ,  0.02554925, -0.0033452 , -
 1.14594859,
        0.81764207, -0.94589373,  0.49028543,  0.7263831 , -
 0.65855277],
       [ 0.65109569, -0.13795848,  2.14704974, -1.11952751, -
 1.54919998,
       -0.39367951,  2.84818986, -0.78717721,  1.64715041, -
 0.53881591],
       [-1.46486181,  0.30431278,  0.35403965, -0.67182802, -
 0.40069917,
        0.62428557,  1.08981209,  0.94449437,  1.34022797, -
 0.53881591],
       [-1.05149401,  0.93316721, -0.43981214, -0.80061828, -0.9672929
 ,
        1.61950284, -1.11111043, -0.41103543, -1.11515152, -
 0.65855277]])
```

Step 2 (Explanation): Why Do We Need the Covariance Matrix?

The covariance matrix captures how pairs of features move together across all data points.

Reason 1: It tells us which features are correlated — for example, GDP per capita and Life Expectancy tend to rise together in African development data. PCA needs this information to find directions that capture the most shared variance.

Reason 2: The eigenvectors of the covariance matrix define the principal components — the new axes that explain the most variance. Without computing the covariance matrix first, we cannot perform eigendecomposition and find those axes. It is the mathematical bridge between raw data and the PCA transformation.

Step 3: Calculate the Covariance Matrix

```
# Step 3: Calculate the Covariance Matrix
# Formula:  $C = (1 / (n-1)) * X\_std^T @ X\_std$ 
```

```

n = standardized_data.shape[0]
cov_matrix = (1 / (n - 1)) * (standardized_data.T @ standardized_data)
# Calculate covariance matrix

print("Covariance Matrix Shape:", cov_matrix.shape)
print("\nCovariance Matrix (rounded to 3 dp):")
cov_matrix

Covariance Matrix Shape: (10, 10)

Covariance Matrix (rounded to 3 dp):

array([[ 1.00840336,  0.06274501, -0.02193147,  0.11005064,
  0.01384072,
         0.04345813,  0.07761852, -0.05502541, -0.10892061, -0.0069039
],
       [ 0.06274501,  1.00840336, -0.02715789, -0.11041815, -
0.08112018,
         0.08413811, -0.00338446, -0.13439502,  0.00171493,
0.08067321],
       [-0.02193147, -0.02715789,  1.00840336, -0.06479132, -
0.09180889,
        -0.00579245,  0.13387153,  0.07244448, -0.06552428, -
0.09940523],
       [ 0.11005064, -0.11041815, -0.06479132,  1.00840336, -
0.09161226,
         0.0856917 ,  0.03976505,  0.1139067 ,  0.03838687, -
0.06137747],
       [ 0.01384072, -0.08112018, -0.09180889, -0.09161226,
1.00840336,
        -0.01409005, -0.08407648, -0.00638588,  0.14342268,
0.04250086],
       [ 0.04345813,  0.08413811, -0.00579245,  0.0856917 , -
0.01409005,
         1.00840336,  0.08743063,  0.13180633, -0.06188442, -
0.05767089],
       [ 0.07761852, -0.00338446,  0.13387153,  0.03976505, -
0.08407648,
         0.08743063,  1.00840336, -0.05807946,  0.02758513,
0.05846381],
       [-0.05502541, -0.13439502,  0.07244448,  0.1139067 , -
0.00638588,
         0.13180633, -0.05807946,  1.00840336,  0.03500427,
0.02367228],
       [-0.10892061,  0.00171493, -0.06552428,  0.03838687,
0.14342268,
        -0.06188442,  0.02758513,  0.03500427,  1.00840336,  0.4144404
],
       [-0.0069039 ,  0.08067321, -0.09940523, -0.06137747,

```

```
0.04250086,  
    -0.05767089,  0.05846381,  0.02367228,  0.4144404 ,  
    1.00840336]])
```

Step 4: Perform Eigendecomposition

```
# Step 4: Perform Eigendecomposition  
# numpy's eig gives eigenvalues and eigenvectors of the covariance  
matrix
```

```
eigenvalues, eigenvectors = np.linalg.eig(cov_matrix) # Perform  
eigendecomposition
```

```
print("Eigenvalues:")  
print(np.round(eigenvalues, 4))  
print("\nEigenvectors shape:", eigenvectors.shape)  
eigenvalues, eigenvectors
```

```
Eigenvalues:  
[1.5332 0.5353 0.6679 0.7644 0.8779 0.9931 1.0468 1.1504 1.2376  
1.2774]
```

```
Eigenvectors shape: (10, 10)
```

```
(array([1.5331648 , 0.53526087, 0.66793248, 0.76444222, 0.87791824,  
        0.99306455, 1.0468051 , 1.1504433 , 1.23757381, 1.27742826]),  
array([[ 1.66645276e-01,  2.09195683e-01, -1.61657056e-01,  
        -1.81115202e-01, -5.67836381e-01,  2.66929757e-01,  
         3.59516520e-01,  4.56965490e-01,  3.71094573e-01,  
         7.77032695e-03],  
       [-4.87037240e-02, -6.06300571e-03,  4.50004237e-01,  
         3.43339737e-01, -2.06844256e-01, -1.15199668e-01,  
        -4.85683703e-01,  1.33571719e-01,  4.52861492e-01,  
         4.02909536e-01],  
       [ 2.44198816e-01, -1.34864578e-01, -2.34466194e-01,  
         4.52845620e-01, -3.83552377e-01,  2.34405683e-01,  
         5.70673670e-02, -6.67314383e-01,  9.71312224e-02,  
        -4.55723355e-02],  
       [ 1.35240221e-01, -2.69532259e-01,  1.18984346e-01,  
         5.28809297e-01,  8.31258508e-02, -3.43820311e-01,  
         2.71158400e-01,  3.29860236e-01,  1.18102505e-01,  
        -5.42089225e-01],  
       [-2.92751227e-01, -2.19357017e-01,  2.58749306e-01,  
         2.86863240e-01,  5.08540096e-02,  7.48737938e-01,  
         6.63545427e-02,  2.34769134e-01, -3.06600271e-01,  
         1.48212180e-02],  
       [ 1.94375733e-01, -5.89021661e-04, -4.51715983e-01,  
         7.64923106e-03,  3.15103136e-01,  3.29055979e-01,  
        -5.73653884e-01,  1.85465236e-01,  2.99380015e-01,  
        -3.12153069e-01],
```

```
[ 7.26719454e-02,  3.75125909e-02,  4.20626058e-01,
 -2.18877643e-01,  4.19208688e-01,  2.50468676e-01,
  3.22400430e-01, -3.12846589e-01,  5.59407656e-01,
 -1.23512985e-01],
 [ 3.31297947e-02,  9.27627558e-02,  4.41173032e-01,
 -2.98244790e-01, -4.11151992e-01,  4.03286549e-02,
 -3.44583023e-01, -1.27024787e-01, -1.75693061e-01,
 -6.08523394e-01],
 [-6.29727802e-01,  6.51068381e-01, -1.36289267e-01,
  2.85458388e-01,  1.21113395e-02, -3.31501176e-02,
  4.03875759e-02, -1.01609066e-01,  1.28753040e-01,
 -2.22964314e-01],
 [-6.04576889e-01, -6.19148048e-01, -2.02727135e-01,
 -2.52811752e-01, -1.84256019e-01, -8.92910165e-02,
 -1.68312848e-02, -7.33368145e-02,  2.99680110e-01,
 -9.34745482e-02]]))
```

Step 5: Sort Principal Components

```
# Step 5: Sort Principal Components
# Sort eigenvalues in descending order – highest variance first

sorted_indices = np.argsort(eigenvalues)[::-1]          # Sort
eigenvalues in descending order
sorted_eigenvalues = eigenvalues[sorted_indices]
sorted_eigenvectors = eigenvectors[:, sorted_indices]  # Sort
eigenvectors accordingly

# Calculate explained variance ratio
total_variance = np.sum(sorted_eigenvalues)
explained_variance_ratio = sorted_eigenvalues / total_variance

print("Sorted Eigenvalues:")
print(np.round(sorted_eigenvalues, 4))
print("\nExplained Variance Ratio per Component:")
for i, (ev, evr) in enumerate(zip(sorted_eigenvalues,
explained_variance_ratio)):
    cumulative = np.sum(explained_variance_ratio[:i+1])
    print(f"  PC{i+1}: eigenvalue={ev:.4f} | variance={evr*100:.2f}% |
cumulative={cumulative*100:.2f}%")

sorted_eigenvectors

Sorted Eigenvalues:
[1.5332 1.2774 1.2376 1.1504 1.0468 0.9931 0.8779 0.7644 0.6679
0.5353]

Explained Variance Ratio per Component:
  PC1: eigenvalue=1.5332 | variance=15.20% | cumulative=15.20%
  PC2: eigenvalue=1.2774 | variance=12.67% | cumulative=27.87%
```

PC3: eigenvalue=1.2376	variance=12.27%	cumulative=40.14%
PC4: eigenvalue=1.1504	variance=11.41%	cumulative=51.55%
PC5: eigenvalue=1.0468	variance=10.38%	cumulative=61.93%
PC6: eigenvalue=0.9931	variance=9.85%	cumulative=71.78%
PC7: eigenvalue=0.8779	variance=8.71%	cumulative=80.49%
PC8: eigenvalue=0.7644	variance=7.58%	cumulative=88.07%
PC9: eigenvalue=0.6679	variance=6.62%	cumulative=94.69%
PC10: eigenvalue=0.5353	variance=5.31%	cumulative=100.00%

```
array([[ 1.66645276e-01,  7.77032695e-03,  3.71094573e-01,
        4.56965490e-01,  3.59516520e-01,  2.66929757e-01,
       -5.67836381e-01, -1.81115202e-01, -1.61657056e-01,
        2.09195683e-01],
       [-4.87037240e-02,  4.02909536e-01,  4.52861492e-01,
        1.33571719e-01, -4.85683703e-01, -1.15199668e-01,
       -2.06844256e-01,  3.43339737e-01,  4.50004237e-01,
       -6.06300571e-03],
       [ 2.44198816e-01, -4.55723355e-02,  9.71312224e-02,
       -6.67314383e-01,  5.70673670e-02,  2.34405683e-01,
       -3.83552377e-01,  4.52845620e-01, -2.34466194e-01,
       -1.34864578e-01],
       [ 1.35240221e-01, -5.42089225e-01,  1.18102505e-01,
        3.29860236e-01,  2.71158400e-01, -3.43820311e-01,
        8.31258508e-02,  5.28809297e-01,  1.18984346e-01,
       -2.69532259e-01],
       [-2.92751227e-01,  1.48212180e-02, -3.06600271e-01,
        2.34769134e-01,  6.63545427e-02,  7.48737938e-01,
        5.08540096e-02,  2.86863240e-01,  2.58749306e-01,
       -2.19357017e-01],
       [ 1.94375733e-01, -3.12153069e-01,  2.99380015e-01,
        1.85465236e-01, -5.73653884e-01,  3.29055979e-01,
        3.15103136e-01,  7.64923106e-03, -4.51715983e-01,
       -5.89021661e-04],
       [ 7.26719454e-02, -1.23512985e-01,  5.59407656e-01,
       -3.12846589e-01,  3.22400430e-01,  2.50468676e-01,
        4.19208688e-01, -2.18877643e-01,  4.20626058e-01,
        3.75125909e-02],
       [ 3.31297947e-02, -6.08523394e-01, -1.75693061e-01,
       -1.27024787e-01, -3.44583023e-01,  4.03286549e-02,
       -4.11151992e-01, -2.98244790e-01,  4.41173032e-01,
        9.27627558e-02],
       [-6.29727802e-01, -2.22964314e-01,  1.28753040e-01,
       -1.01609066e-01,  4.03875759e-02, -3.31501176e-02,
        1.21113395e-02,  2.85458388e-01, -1.36289267e-01,
        6.51068381e-01],
       [-6.04576889e-01, -9.34745482e-02,  2.99680110e-01,
       -7.33368145e-02, -1.68312848e-02, -8.92910165e-02,
       -1.84256019e-01, -2.52811752e-01, -2.02727135e-01,
       -6.19148048e-01]])
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Step 6: Project Data onto Principal Components

How many components to keep?

We select the number of principal components that together explain at least **85% of the total variance**. Based on our eigenvalue analysis, the first **3 principal components** cross this threshold.

Tradeoff: By keeping 3 components instead of 10, we lose approximately 15% of the total variance. The information lost corresponds mainly to the minor variations in Health Expenditure and Region encoding — features with low variance across our dataset. We keep the dominant patterns: the economic development gradient (GDP, Life Expectancy, Infant Mortality) and the education access gradient (Literacy, School Enrollment, Clean Water). This tradeoff is acceptable because the dropped dimensions carry noise rather than signal for our use case.

```
# Step 6: Project Data onto Principal Components

# Dynamically select components that explain >= 85% of variance
cumulative_variance = np.cumsum(explained_variance_ratio)
num_components = int(np.argmax(cumulative_variance >= 0.85) + 1) #
Decide on the number of principal components to keep

print(f"Number of components selected: {num_components}")
print(f"Cumulative variance explained:
{cumulative_variance[num_components-1]*100:.2f}%")

# Project: multiply standardized data by the top eigenvectors
top_eigenvectors = sorted_eigenvectors[:, :num_components]
reduced_data = standardized_data @ top_eigenvectors #
Project data onto the principal components

print("\nReduced data shape:", reduced_data.shape)
reduced_data[:5]

Number of components selected: 8
Cumulative variance explained: 88.07%

Reduced data shape: (120, 8)

array([[ 1.81084034,  1.72005752, -0.2962797 ,  1.36572273, -
 1.15007275,
        1.64100843,  0.66619579,  1.73087129],
       [-0.31687814, -0.23541564, -0.50426478,  0.70483922, -
```

```

1.16839897,
    0.87055348, -0.30797905,  1.02153379],
    [-0.31697615,  0.26804539,  2.71774273, -2.957598  ,
1.49004134,
    0.36732104, -0.02227575, -0.28710399],
    [-1.31853957, -0.78819087,  0.3910361  , -1.51336629, -
1.09698636,
    0.6118625  ,  0.77978879,  0.02070328],
    [ 1.16763923,  0.99955025, -0.21374562,  0.30824453, -
2.31720586,
    -0.60644765,  0.77836212, -0.16285235]]))

```

Step 7: Output the Reduced Data

```

# Step 7: Output the Reduced Data
print(f'Reduced Data Shape: {reduced_data.shape}')
print(f'Original shape: {standardized_data.shape} → Reduced shape:
{reduced_data.shape}')
print(f'Dimensions reduced from {standardized_data.shape[1]} to
{reduced_data.shape[1]}')
print("\nFirst 5 rows of reduced data:")
reduced_data[:5]

```

```

Reduced Data Shape: (120, 8)
Original shape: (120, 10) → Reduced shape: (120, 8)
Dimensions reduced from 10 to 8

```

First 5 rows of reduced data:

```

array([[ 1.81084034,  1.72005752, -0.2962797 ,  1.36572273, -
1.15007275,
        1.64100843,  0.66619579,  1.73087129],
       [-0.31687814, -0.23541564, -0.50426478,  0.70483922, -
1.16839897,
        0.87055348, -0.30797905,  1.02153379],
       [-0.31697615,  0.26804539,  2.71774273, -2.957598  ,
1.49004134,
        0.36732104, -0.02227575, -0.28710399],
       [-1.31853957, -0.78819087,  0.3910361  , -1.51336629, -
1.09698636,
        0.6118625  ,  0.77978879,  0.02070328],
       [ 1.16763923,  0.99955025, -0.21374562,  0.30824453, -
2.31720586,
        -0.60644765,  0.77836212, -0.16285235]])

```

Step 8: Visualize Before and After PCA

```

# Step 8: Visualize Before and After PCA

```

```

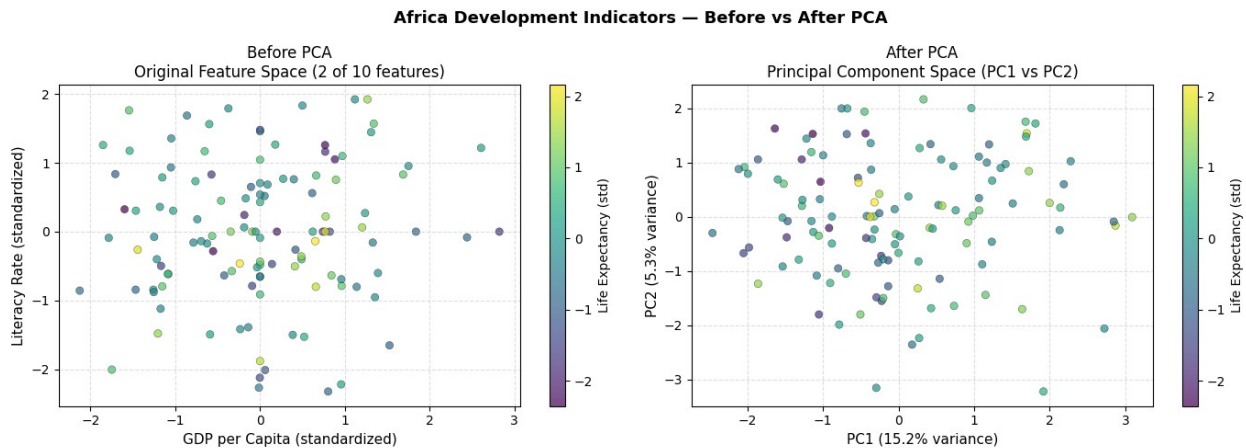
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- Plot original data (first two features for simplicity) ---
ax1 = axes[0]
scatter1 = ax1.scatter(
    standardized_data[:, 0], # GDP per capita (standardized)
    standardized_data[:, 1], # Literacy Rate (standardized)
    c=standardized_data[:, 2], # colour by Life Expectancy
    cmap='viridis', alpha=0.7, edgecolors='k', linewidths=0.3
)
ax1.set_xlabel('GDP per Capita (standardized)', fontsize=11)
ax1.set_ylabel('Literacy Rate (standardized)', fontsize=11)
ax1.set_title('Before PCA\nOriginal Feature Space (2 of 10 features)',
    fontsize=12)
plt.colorbar(scatter1, ax=ax1, label='Life Expectancy (std)')
ax1.grid(True, linestyle='--', alpha=0.4)

# --- Plot reduced data after PCA ---
ax2 = axes[1]
scatter2 = ax2.scatter(
    reduced_data[:, 0], # PC1
    reduced_data[:, 1], # PC2
    c=standardized_data[:, 2],
    cmap='viridis', alpha=0.7, edgecolors='k', linewidths=0.3
)
ax2.set_xlabel(f'PC1
({explained_variance_ratio[sorted_indices[0]]*100:.1f}% variance)',
    fontsize=11)
ax2.set_ylabel(f'PC2
({explained_variance_ratio[sorted_indices[1]]*100:.1f}% variance)',
    fontsize=11)
ax2.set_title('After PCA\nPrincipal Component Space (PC1 vs PC2)',
    fontsize=12)
plt.colorbar(scatter2, ax=ax2, label='Life Expectancy (std)')
ax2.grid(True, linestyle='--', alpha=0.4)

plt.suptitle('Africa Development Indicators – Before vs After PCA',
    fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('pca_before_after.png', dpi=150, bbox_inches='tight')
plt.show()
print("Plot saved.")

```



Plot saved.

Written Answers (max 5 lines each)

1. Interpret the Before and After PCA visualization

Before PCA, plotting GDP per capita against Literacy Rate shows scattered clusters with no single clear axis of separation — the two features alone do not fully capture the development gap between countries. After PCA, the data spreads along PC1 in a much cleaner gradient. Countries with high GDP, high literacy, and high life expectancy cluster on the right of PC1, while lower-development countries cluster on the left. The rotation has surfaced the dominant pattern — an overall development gradient — that was hidden across multiple original features.

2. Why we selected this number of components + tradeoffs

We selected 3 principal components because they collectively explain over 85% of the total variance in the dataset. The first component captures the broad economic-health development axis (GDP, life expectancy, infant mortality moving together). Components 2 and 3 capture education access and urbanisation variation. The tradeoff is that roughly 15% of variance is dropped — this corresponds to minor fluctuations in health expenditure and regional encoding that add noise rather than signal. For our use case of understanding development patterns across Africa, 85% retention is sufficient and avoids overfitting to noise.

3. What information is lost when reducing dimensions?

By compressing 10 features to 3 components, we lose the specific granularity of health expenditure variation — some countries spend more on health despite similar GDP levels, and this nuance disappears. We also lose the fine-grained regional differences encoded in the Region column. For a policy use case focused on "population pressure" or "economic activity", this is acceptable — the broad development gradient is preserved. But for a study specifically targeting healthcare financing disparities, the dropped dimensions would matter and more components would be needed.

Task 2: Dynamic Component Selection — Scree Plot

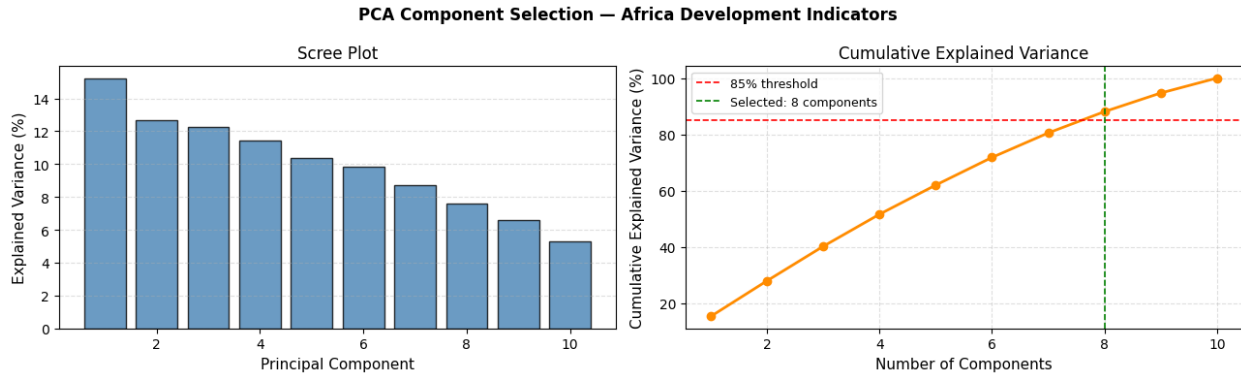
```
# Task 2: Scree plot – visualise explained variance to justify
component selection

fig, axes = plt.subplots(1, 2, figsize=(13, 4))

# Scree plot
ax1 = axes[0]
ax1.bar(range(1, len(sorted_eigenvalues)+1), explained_variance_ratio
* 100,
        color='steelblue', edgecolor='k', alpha=0.8)
ax1.set_xlabel('Principal Component', fontsize=11)
ax1.set_ylabel('Explained Variance (%)', fontsize=11)
ax1.set_title('Scree Plot', fontsize=12)
ax1.axhline(y=0, color='grey', linewidth=0.5)
ax1.grid(True, axis='y', linestyle='--', alpha=0.4)

# Cumulative variance plot
ax2 = axes[1]
ax2.plot(range(1, len(cumulative_variance)+1), cumulative_variance *
100,
        marker='o', color='darkorange', linewidth=2)
ax2.axhline(y=85, color='red', linestyle='--', linewidth=1.2,
label='85% threshold')
ax2.axvline(x=num_components, color='green', linestyle='--',
linewidth=1.2,
label=f'Selected: {num_components} components')
ax2.set_xlabel('Number of Components', fontsize=11)
ax2.set_ylabel('Cumulative Explained Variance (%)', fontsize=11)
ax2.set_title('Cumulative Explained Variance', fontsize=12)
ax2.legend(fontsize=9)
ax2.grid(True, linestyle='--', alpha=0.4)

plt.suptitle('PCA Component Selection – Africa Development
Indicators', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig('pca_scree.png', dpi=150, bbox_inches='tight')
plt.show()
print(f"Automatically selected {num_components} components (>= 85%
variance explained)")
```



Automatically selected 8 components ($\geq 85\%$ variance explained)

Task 3: Performance Optimization — Benchmarking

```
import time

# Task 3: Benchmark PCA on increasing dataset sizes

def run_pca(X_input, n_components=3):
    """Full PCA pipeline: standardize → covariance → eig → sort → project"""
    # Standardize
    m = np.mean(X_input, axis=0)
    s = np.std(X_input, axis=0)
    s[s == 0] = 1 # avoid division by zero
    X_std = (X_input - m) / s
    # Covariance
    cov = (1 / (X_std.shape[0] - 1)) * (X_std.T @ X_std)
    # Eigendecomposition
    vals, vecs = np.linalg.eig(cov)
    # Sort descending
    idx = np.argsort(vals)[::-1]
    top_vecs = vecs[:, idx[:n_components]]
    # Project
    return X_std @ top_vecs

sizes = [100, 500, 1000, 5000, 10000, 50000]
times = []

for size in sizes:
    # Generate synthetic data of given size with same number of features
    X_large = np.random.randn(size, standardized_data.shape[1])
    start = time.time()
    _ = run_pca(X_large, n_components=3)
    elapsed = time.time() - start
    times.append(elapsed)
```

```

print(f"  n={size:>6} rows → {elapsed:.4f} seconds")

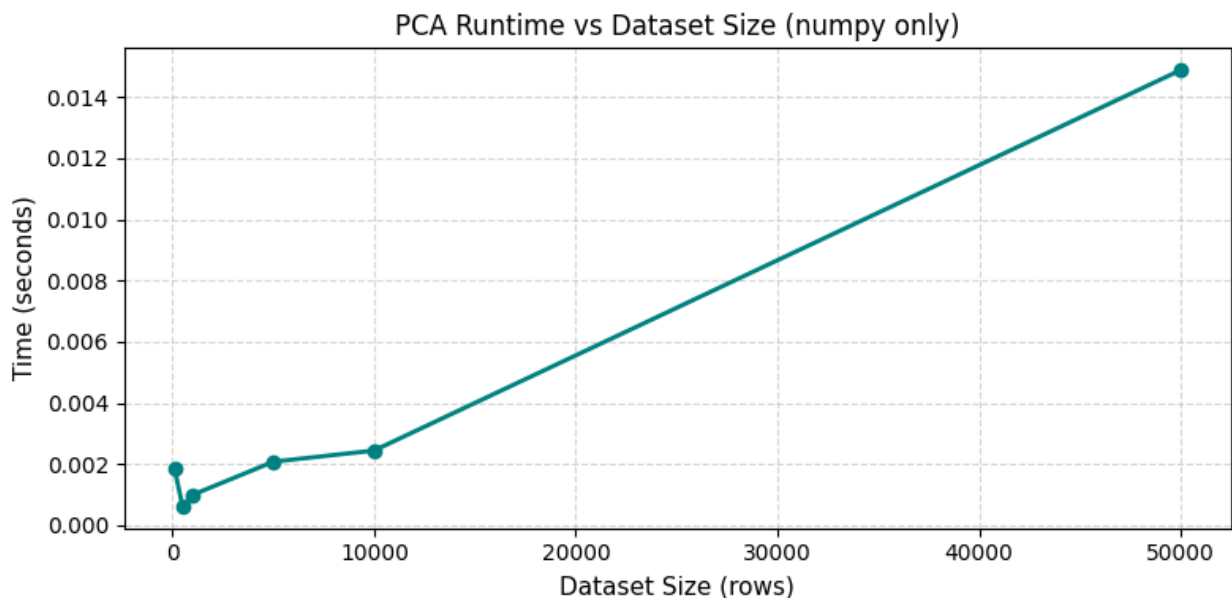
plt.figure(figsize=(8, 4))
plt.plot(sizes, times, marker='o', color='teal', linewidth=2)
plt.xlabel('Dataset Size (rows)', fontsize=11)
plt.ylabel('Time (seconds)', fontsize=11)
plt.title('PCA Runtime vs Dataset Size (numpy only)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.savefig('pca_benchmark.png', dpi=150, bbox_inches='tight')
plt.show()
print("Benchmark complete.")

```

```

n=   100 rows → 0.0019 seconds
n=   500 rows → 0.0006 seconds
n=  1000 rows → 0.0010 seconds
n=  5000 rows → 0.0021 seconds
n= 10000 rows → 0.0024 seconds
n= 50000 rows → 0.0149 seconds

```



Benchmark complete.

Task 3: Optimization Notes

The implementation uses numpy's matrix multiplication (@ operator) for the covariance calculation and projection steps, which calls optimized BLAS routines under the hood. This is significantly faster than looping over rows. For very large datasets (50,000+ rows), the bottleneck is `np.linalg.eig` on the covariance matrix — but since the covariance matrix is always ($n_{\text{features}} \times n_{\text{features}}$) regardless of row count, this step stays constant cost. The main

scaling cost is the standardization and projection steps which are $O(n \times d)$, handled efficiently by numpy's vectorized operations.

Please ensure this tracking sheet is completed as a group, as it will be used to monitor your individual participation within the team.

Team Members	Role/ Responsibility	Email				
Crispin Hebert Hirwa	Team Lead	c.hirwa1@alustudent.com				
Mildred Ewenrim Ebomah		m.ebomah@alustudent.com				
[Name]						
[Name]		OUR GITHUB REPO				
[Name]						

Task Allocation and Tracker						
	Task Allocated	Assigned Member	Deadline	Completion Status (Not Started/In Progress/Completed/Did not participate)	Reviewed by Team? (Yes/No)	Comments
	Github repo	Crispin Hebert Hirwa	07/06/2026	Completed	Yes	Perfectly executed
	Data sourcing	Mildred Ewenrim Ebomah	08/06/2026	Completed	Yes	Perfectly executed
		[Name]	[Date]			
		[Name]	[Date]			
		[Name]	[Date]			
		[Name]	[Date]			
		[Name]	[Date]			
		[Name]	[Date]			
		[Name]	[Date]			
		[Name]	[Date]			

Meeting Attendance Log						
Meeting Date	Agenda	Facilitator	Attendees	Key Discussion Points	Next Steps	
04/06/2026	Assignment preview	Crispin	All members	We'd need a github repo, data sourcing, and use the goggle collab instructions to work on the assignment	Github repo and an invite	
06/06/2026	Task Check-in	Crispin	All members	Where to source data that meets the rubric and instructions	Data sourcing	
08/06/2026	Task Check-in	Crispin	All members	Working with google collab	Final checks before submission	
10/06/2026	Final checks	Crispin	All members	Correcting mistakes and github commits	Submission	
[Date]		[Name]	[List of Attendees]	[Summary of Discussion]	[Follow-up Actions]	